Complete Search, Greedy Algorithms and Dynamic Programming

AIL7002 · Data Structures and Algorithms (refresher) · IIT Delhi
Based on Chapters 5–7 of A. Laaksonen, Competitive Programmer's Handbook



Three ways to attack the same problem

Complete search: generate every candidate solution, keep the best (or count them). Always correct; fast enough only when the search space is small.
Greedy: build the solution by always taking the locally best choice and never undoing it. Very fast; correct only for the right rule.
Dynamic programming: complete search where each distinct subproblem is solved once and stored. Correct like complete search, efficient like greedy.



Same decision tree, three strategies: explore all branches · follow one branch · fuse repeated subproblems into a small DAG

PART 1

Part 1 · Complete search

Complete search: try everything

Generate all possible solutions by brute force; pick the best one, or count them.
Easy to write, and always gives the correct answer.
Works when there is enough time to go through all solutions — otherwise we need greedy or DP.
Rule of thumb: a modern computer does a few hundred million simple operations per second. Use the input size to guess the complexity you can afford (time limit ≈ 1 s):

Input size	Affordable complexity	Typical complete search
$n \le 10$	$O(n!)$	all permutations
$n \le 20$	$O(2^n)$	all subsets
$n \le 500$	$O(n^3)$	all triples
$n \le 5000$	$O(n^2)$	all pairs
$n \le 10^6$	$O(n \log n)$ or $O(n)$	—

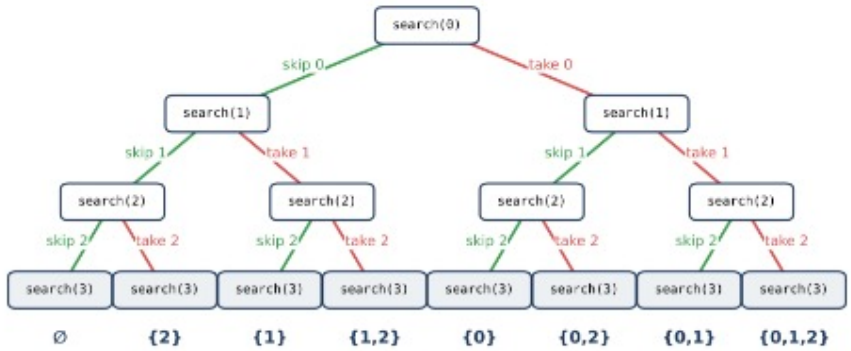
How big is too big? (recap)

n	2^n operations	time at 10^9 operations/s
30	$\approx 1.1 \times 10^9$	≈ 1 second
50	$\approx 1.1 \times 10^{15}$	≈ 13 days
60	$\approx 1.2 \times 10^{18}$	≈ 37 years

Two ways out - Prune the search: skip branches that cannot lead to a (better) solution — backtracking. - Split the search: two searches of size $2^{n/2}$ instead of one of size 2^n — meet in the middle.

Generating subsets (method 1): recursion

```
void search(int k) {
 if (k == n) {
   // process subset
 } else {
   search(k+1); // k NOT in subset
   subset.push_back(k);
   search(k+1); // k in subset
   subset.pop_back();
 }
}
```



Call search(0). Each call decides one element: skip it (left) or take it (right).
2^n leaves, one per subset; $2^{n+1} - 1$ calls in total.

Call tree for n = 3: green = skip, red = take; the leaves list all 8 subsets of {0,1,2}

Generating subsets (method 2): bitmasks

```
for (int b = 0; b < (1<<n); b++) {
  vector<int> subset;
  for (int i = 0; i < n; i++) {
   if (b&(1<<i)) subset.push_back(i);
  }
}
```

b	bits (2 1 0)	subset
0	0 0 0	∅
1	0 0 1	{0}
2	0 1 0	{1}
3	0 1 1	{0,1}
4	1 0 0	{2}
5	1 0 1	{0,2}
6	1 1 0	{1,2}
7	1 1 1	{0,1,2}

Example: b = 25

1 1 0 0 1
4 3 2 1 0

element i ∈ subset ⇔ b & (1<<i) ≠ 0

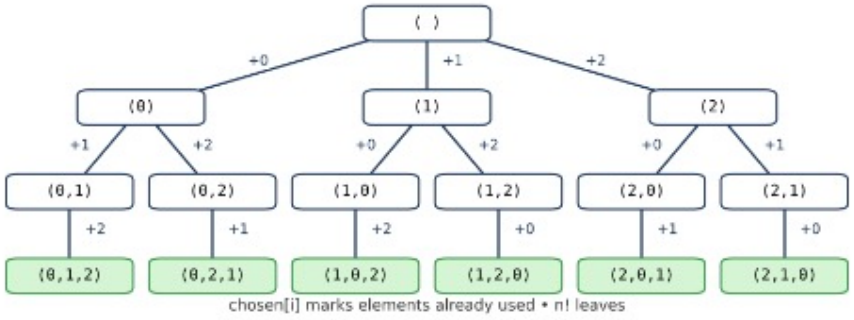
$25 = 11001_2 \rightarrow \{0, 3, 4\}$

A subset of {0,…,n−1} ↔ an n-bit integer b in 0 … $2^n - 1$; bit i set ⇔ element i is in the subset.
No recursion; very fast in practice. Works for n up to about 20 (fits in an int up to 31).

Subsets of {0,1,2} as bitmasks; $25 = 11001_2$ represents {0, 3, 4}

Generating permutations

```
void search() {
    if (permutation.size() == n) {
        // process permutation
    } else {
        for (int i = 0; i < n; i++) {
            if (chosen[i]) continue;
            chosen[i] = true;
            permutation.push_back(i);
            search();
            chosen[i] = false;
            permutation.pop_back();
        }
    }
}
```

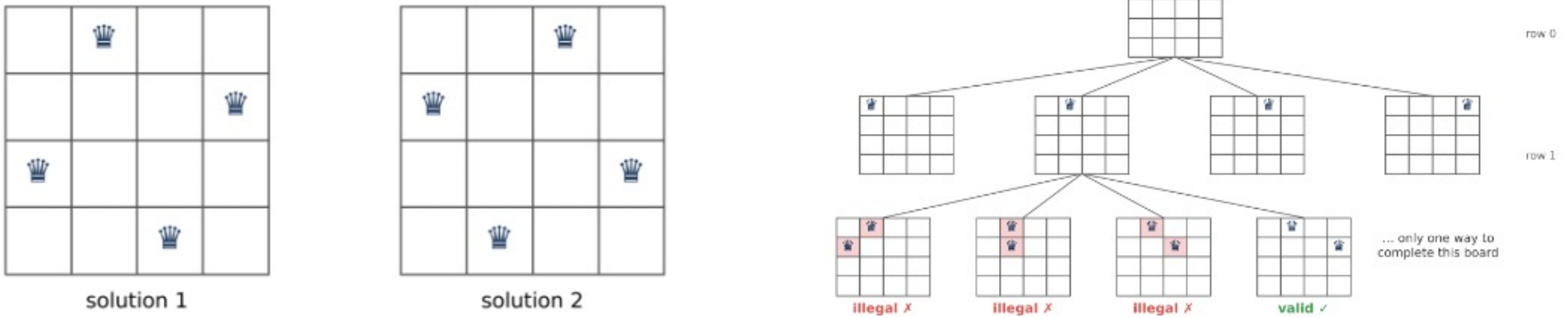


Alternative (C++): start from the sorted permutation and loop with **next_permutation(begin, end)** — produces all n! permutations in increasing order.

Recursion tree for n = 3: each level appends one unused element; 3! = 6 leaves

Backtracking: the n-queens problem

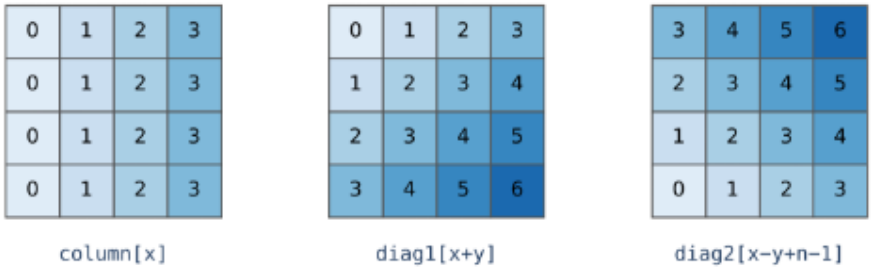
Backtracking starts with an empty solution and extends it step by step, abandoning a partial solution as soon as it cannot be completed.
Problem: count the ways to place n queens on an n × n board so that no two attack each other.
Strategy: place exactly one queen per row, row by row; only try squares not attacked by queens already placed.



The two solutions for n = 4 · Partial search tree for n = 4: with the first queen in column 1, three placements in row 1 are illegal and are cut off immediately; only one can be extended

n-queens: the code

```
void search(int y) {
    if (y == n) {
        count++;
        return;
    }
    for (int x = 0; x < n; x++) {
        if (column[x] || diag1[x+y] || diag2[x-y+n-1]) continue;
        column[x] = diag1[x+y] = diag2[x-y+n-1] = 1;
        search(y+1);
        column[x] = diag1[x+y] = diag2[x-y+n-1] = 0;
    }
}
```

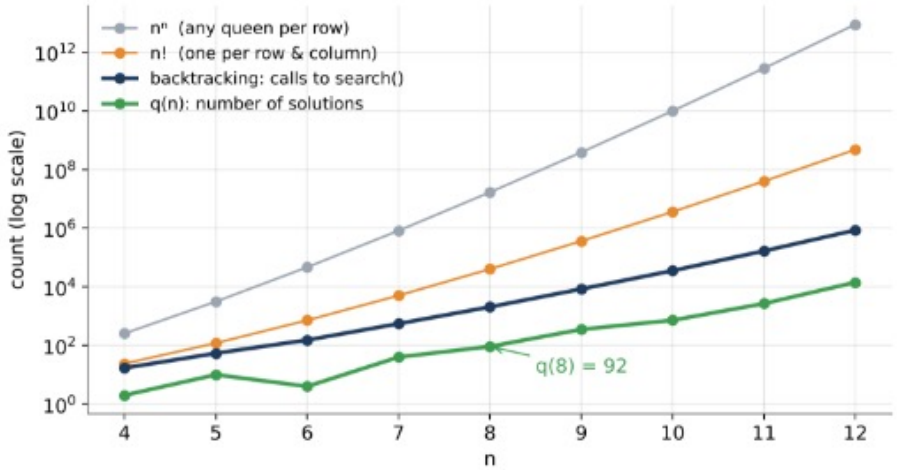


Three boolean arrays make the "is this square attacked?" test O(1): squares on the same anti-diagonal share x + y; on the same diagonal they share x − y.

Index of the column, diag1 and diag2 arrays for each square of a 4 × 4 board

How well does the pruning work?

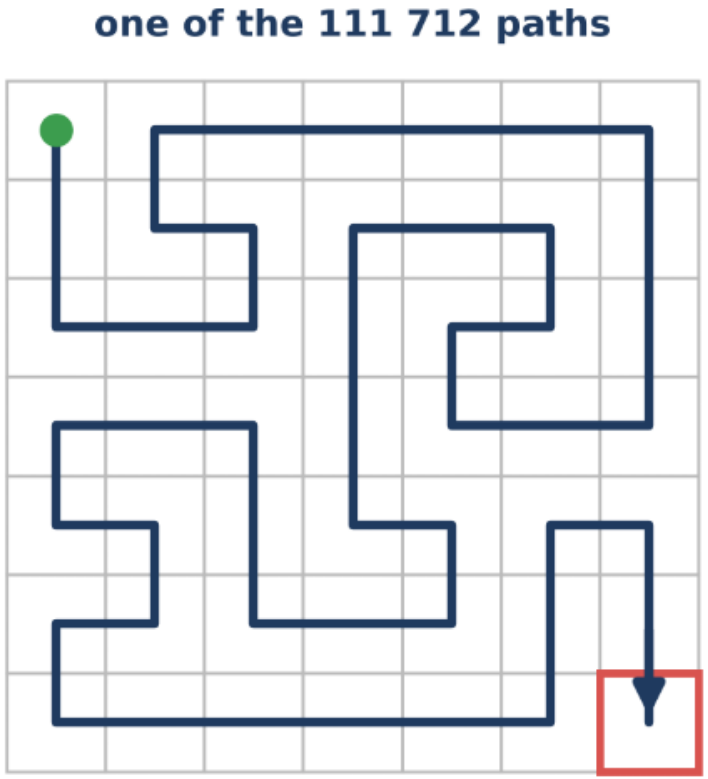
q(8) = 92. For n = 12 the search makes about 8.6×10^5 calls, versus $12! \approx 4.8 \times 10^8$ permutations. Still exponential: the book reports q(16) = 14 772 512 takes about a minute.



Calls to search() versus n^n and n!, and the number of solutions q(n), for n = 4 … 12 (log scale)

Pruning case study: paths through a 7 × 7 grid

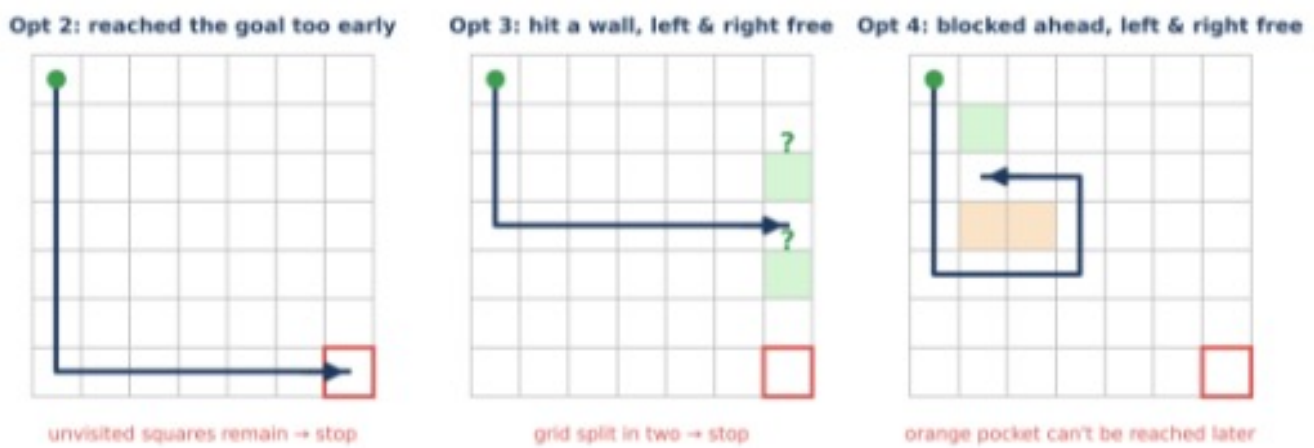
Count paths from the upper-left to the lower-right square that visit every square exactly once (moves: up, down, left, right).
For 7 × 7 there are 111 712 such paths.
Plain backtracking: extend the path square by square, never revisiting a square.



One valid path (green dot = start, red square = goal)

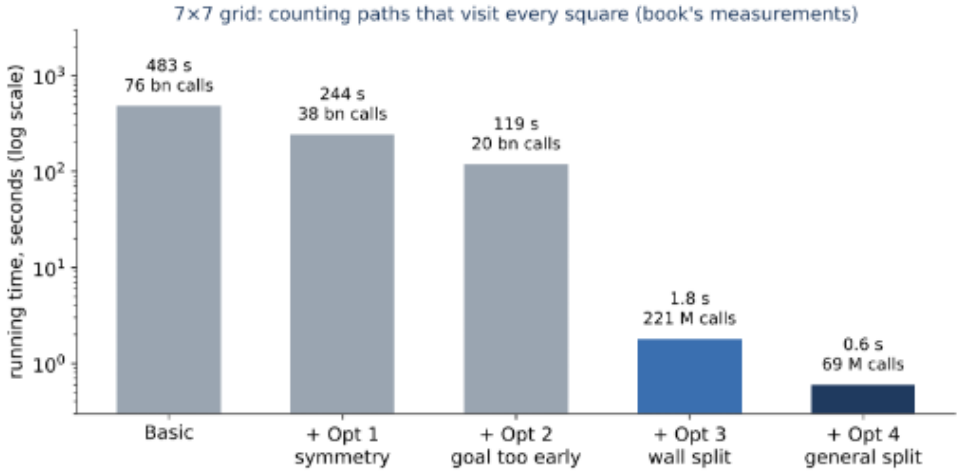
Four observations that prune the search

Opt 1 — symmetry: the first move is down or right, and the two cases are mirror images. Always move down first and double the count.
Opt 2 — goal too early: if the path reaches the lower-right square before visiting every square, stop.
Opt 3 — wall split: if the path hits a wall and could turn left or right, the unvisited squares are split into two parts; stop.
Opt 4 — general split: if the path cannot go forward but both left and right are free, the grid splits; stop.



Dead ends detected by optimisations 2, 3 and 4

Pruning pays off — about 1000× faster

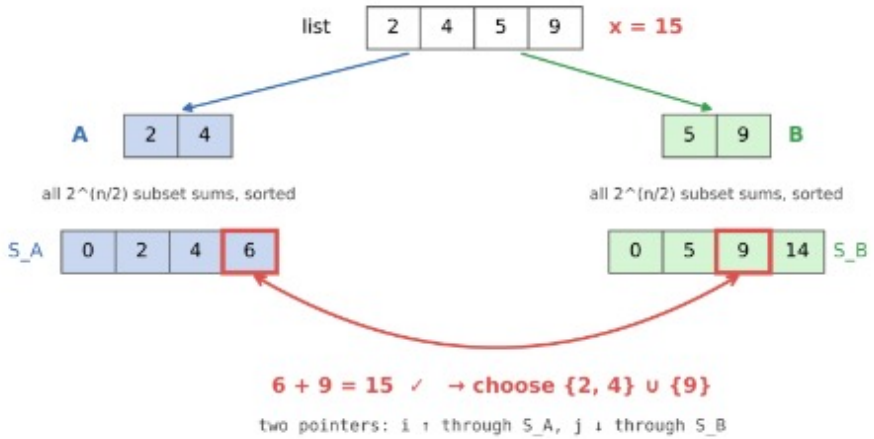


Pruning toolkit (recap) - Constraints: abandon a branch that already violates a constraint (queens). - Feasibility look-ahead: abandon a branch that can no longer be completed (grid split). - Bounds: abandon a branch whose best possible value cannot beat the best found so far (branch-and-bound). - Prune early: cuts near the top of the search tree save the most.

Running time and number of recursive calls after each optimisation (measurements reported in the book)

Meet in the middle

Problem: given n numbers and a target x, can we choose a subset whose sum is x? Complete search: 2^n subsets.
Idea: split the list into halves A and B; enumerate all subset sums of each half ($2^{n/2}$ each); check whether some s_A + s_B = x.
Example: [2, 4, 5, 9], x = 15 → A = [2, 4], B = [5, 9]; S_A = [0, 2, 4, 6], S_B = [0, 5, 9, 14]; 6 + 9 = 15 ✓ → choose {2, 4, 9}.
For x = 10 no pair works, so the sum cannot be formed.



Split, enumerate each half, then match one sum from each side

PART 2

Part 2 · Greedy algorithms

Greedy algorithms

Build the solution by always making the choice that looks best right now.
Never take a choice back ⇒ usually very efficient (often a sort plus a linear scan).
The hard part is not coding but correctness: the locally optimal choices must also be globally optimal.
Two tools:

- a counterexample shows a greedy rule is wrong;
- an exchange argument shows it is right.

Greedy vs. backtracking (recap) Both break the problem into a sequence of local decisions. Backtracking tries every option for each decision; greedy commits to one option per decision.

Coin problem: the greedy rule

Coins c_1, ..., c\ k (unlimited supply of each); form the sum n with as few coins as possible.
Greedy: repeatedly take the largest coin that does not exceed the remaining sum.
Euro coins {1, 2, 5, 10, 20, 50, 100, 200} and n = 520: 200 + 200 + 100 + 20 — four coins, which is optimal.



Greedy is optimal for euro coins but fails for coins {1, 3, 4} and n = 6

When does the coin greedy work?

Euro coins: in an optimal solution, 1, 5, 10, 50, 100 appear at most once (5 + 5 → 10), 2 and 20 at most twice (2 + 2 + 2 → 5 + 1), and 2 + 2 + 1, 20 + 20 + 10 never appear (→ 5, 50).

Hence the smaller coins can never optimally form a sum ≥ the next coin (e.g. the most they can optimally form below 100 is 50 + 20 + 20 + 5 + 2 + 2 = 99), so taking the largest coin is always safe.

General coin sets: greedy may fail. {1, 3, 4}, n = 6: greedy 4 + 1 + 1, optimal 3 + 3.

No greedy rule is known for the general problem — but dynamic programming solves it (Part 3).

Scheduling: as many events as possible

Event	Start	End
A	1	3
B	2	5
C	3	9
D	6	8



Given n events with start and end times, choose the maximum number of non-overlapping events.
The maximum is 2 events (e.g. B and D, or A and D).

Greedy by earliest end time: A ✓, B ✗ (overlaps A), D ✓, C ✗ → 2 events

Which greedy rule? Two that fail

Rule 1 — shortest event first: fails when a short event overlaps two long, compatible ones.
Rule 2 — earliest start first: fails when the first event is very long.
Rule 3 — earliest end first: always optimal (next slide).



Counterexamples: the greedy choice (filled) blocks two events that could both be chosen

Earliest end first — why it works

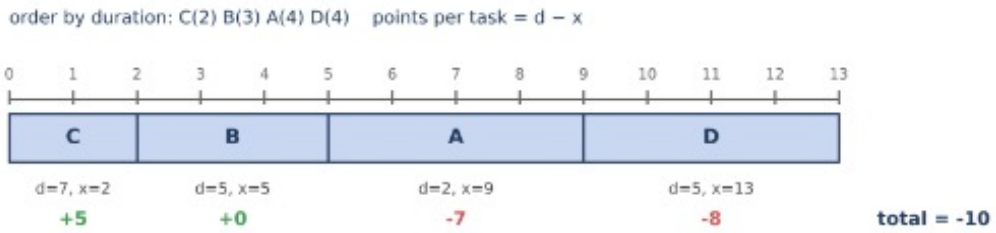
Exchange argument: take any optimal schedule and replace its first event by the event that ends earliest. It still fits (it ends no later), and the number of events is unchanged. Repeat for the rest.

So choosing the earliest-ending event never hurts, and the greedy schedule is optimal.

O(n log n) for the sort, O(n) for the scan.

Tasks and deadlines

Task	Duration	Deadline
A	4	2
B	3	5
C	2	7
D	4	5



n tasks, each with a duration and a deadline d. Finishing a task at time x earns d − x points (can be negative). Maximise the total.

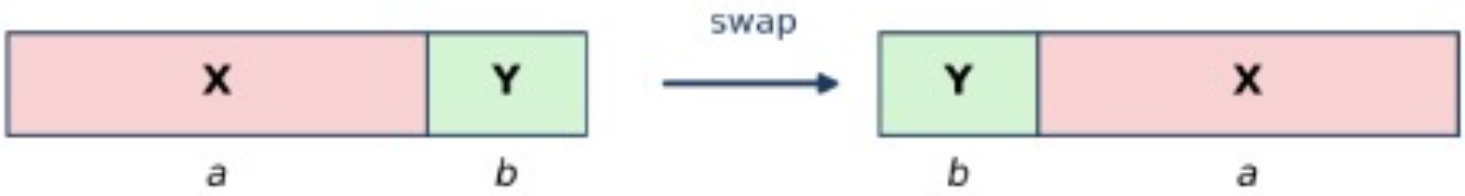
Optimal order C, B, A, D: 5 + 0 − 7 − 8 = −10.

Surprise: the best order does not depend on the deadlines at all — just do the tasks in increasing order of duration.

Schedule sorted by duration and the points of each task

The exchange argument

If a longer task X is immediately followed by a shorter task Y, swap them: Y finishes a time units earlier, X finishes b units later; the total improves by a − b > 0.
So in an optimal order no longer task precedes a shorter one ⇒ sort by duration.

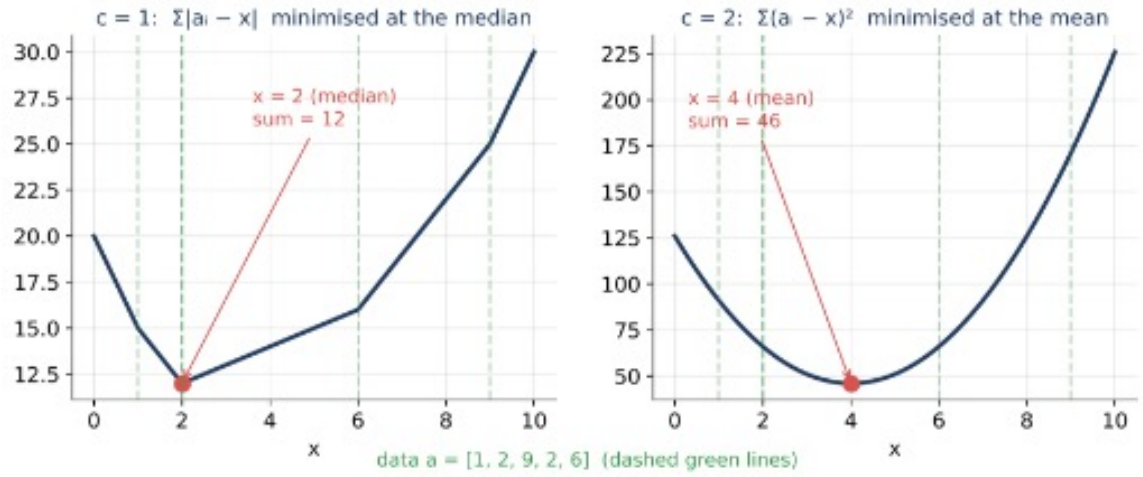


a > b: Y finishes a earlier (+a), X finishes b later (−b) ⇒ total improves by a − b > 0

Exchange-argument template Take an optimal solution that differs from the greedy one; swap or replace one element to make it agree with greedy without making it worse; repeat. *Swapping two adjacent tasks X (length a) and Y (length b) with a > b*

Minimising sums: median and mean

Given $a_1, \ldots, a_n$, find x minimising $\Sigma\, |a_i - x|^c$.
c = 1: choose the median. Moving x towards the median always decreases the sum.
Example [1, 2, 9, 2, 6]: sorted [1, 2, 2, 6, 9], x = 2, sum = 12.
c = 2: choose the mean. $\Sigma(a_i - x)^2 = n\,x^2 - 2x\,s + \Sigma\, a_i^2$ with $s = \Sigma\, a_i$ is a parabola minimised at x = s/n. Example: x = 4, sum = 46.
For even n, any x between the two middle values is optimal for c = 1.



The cost as a function of x for c = 1 (piecewise linear) and c = 2 (parabola)

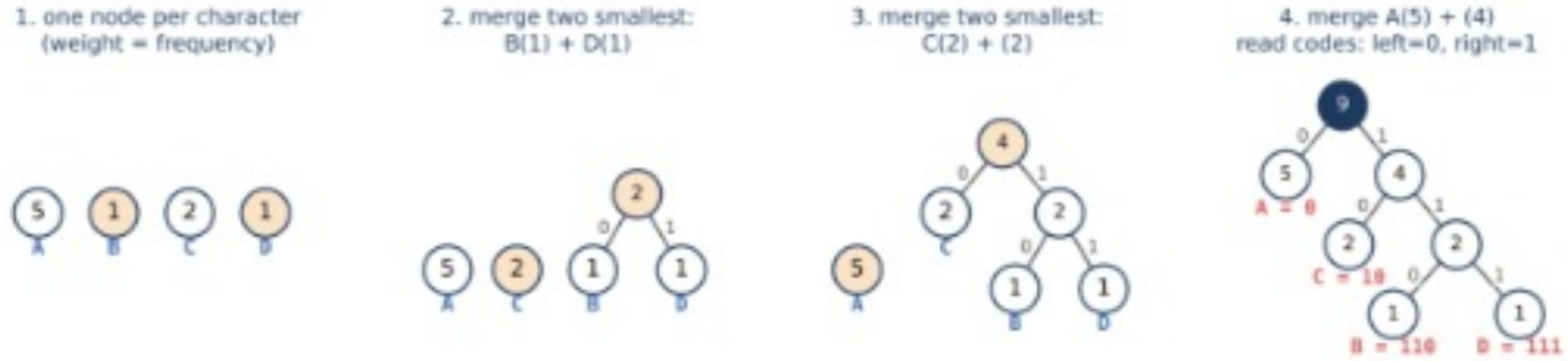
Data compression: binary codes

A binary code gives each character a codeword of bits. String AABACDACA (9 characters):
Frequent characters get short codewords.
Prefix-free: no codeword may be a prefix of another — otherwise decoding is ambiguous (with A = 10, B = 11, C = 1011, the bits 1011 could mean AB or C).

Character	Constant-length code	Optimal variable-length code
A (5×)	00	0
B (1×)	01	110
C (2×)	10	10
D (1×)	11	111
Total bits	18	15

Huffman coding

Start with one node per character, weighted by its frequency.
Repeatedly merge the two nodes of smallest weight into a new node whose weight is their sum, until one tree remains.
Read each codeword from the root: left edge = 0, right edge = 1.
With a priority queue: O(k log k) for k distinct characters.



Huffman coding for AABACDACA: A = 0, C = 10, B = 110, D = 111 (15 bits)

Designing a greedy algorithm — checklist

Practical tip: write the complete-search solution first (Part 1), then compare it with the greedy answer on thousands of random small inputs. It finds wrong greedy rules in seconds.

Checklist

1. **Propose a rule:** sort by something, or always take the min / max.
2. **Try to break it on tiny inputs** — by hand and with a brute-force checker.
3. **If it survives**, prove it with an **exchange argument**.
4. **Implement:** usually a sort or a priority queue, O(n log n).

PART 3

Part 3 · Dynamic programming

Dynamic programming

Applies when a problem divides into overlapping subproblems that can be solved independently.
Two uses: **find an optimal solution** (min / max), or **count the solutions.**

Recipe: write the complete search as a recursive function of a small state, then store each answer so it is computed only once (memoization).

Coin problem revisited: a recursive formulation

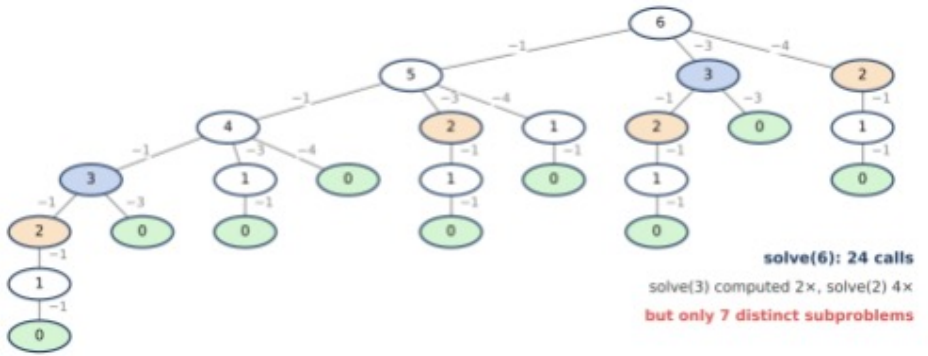
solve(x) = ∞ if x < 0
solve(x) = minimum number of coins needed to form the sum x. Focus on the first coin chosen:
Coins {1, 3, 4}:
solve(10) = solve(7) + 1 = solve(4) + 2 = solve(0) + 3 = 3 (10 = 3 + 3 + 4).

x	0	1	2	3	4	5	6	7	8	9	10
solve(x)	0	1	2	1	1	2	2	2	2	3	3

$$\begin{aligned} solve(x) = \min(&solve(x-1)+1,\\ &solve(x-3)+1,\\ &solve(x-4)+1). \end{aligned}$$

Direct recursion repeats work

```
int solve(int x) {
    if (x < 0) return INF;
    if (x == 0) return 0;
    int best = INF;
    for (auto c : coins) {
        best = min(best, solve(x-c)+1);
    }
    return best;
}
```



The number of calls grows exponentially: counting also the calls with negative arguments, solve(10) makes 313 calls, solve(20) 38 446 and solve(30) about 4.7 million.

Call tree of solve(6) with coins {1, 3, 4} (negative arguments omitted): 24 calls but only 7 distinct subproblems; repeated solve(3) and solve(2) are highlighted

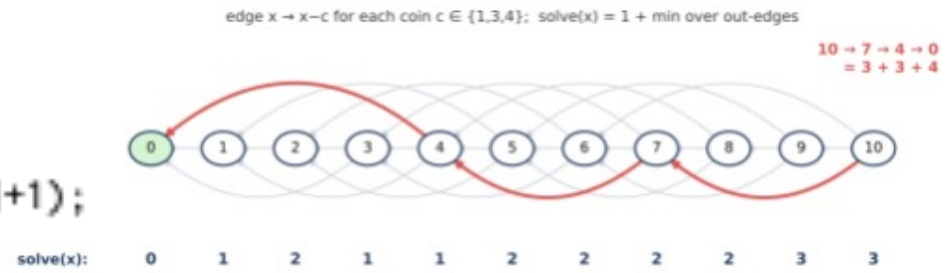
Memoization

```
int solve(int x) {
    if (x < 0) return INF;
    if (x == 0) return 0;
    if (ready[x]) return value[x];
    int best = INF;
    for (auto c : coins) {
        best = min(best, solve(x-c)+1);
    }
    value[x] = best;
    ready[x] = true;
    return best;
}
```

Each value solve(x) is computed recursively only once, then looked up. Time O(n k) for target sum n and k coins.

Bottom-up: filling the table in order

```
value[0] = 0;
for (int x = 1; x <= n; x++) {
    value[x] = INF;
    for (auto c : coins) {
        if (x-c >= 0) {
            value[x] = min(value[x], value[x-c]+1);
        }
    }
}
```



Solve smaller sums first, so everything solve(x) needs is ready. Shorter code, smaller constant factors — the form most competitive programmers write.

The subproblems and their dependencies form a DAG; any order that respects the arrows (here x = 0, 1, 2, ...) works.

Subproblem DAG for coins {1, 3, 4}: each x points to x − 1, x − 3, x − 4; the optimal choice for 10 is highlighted

Top-down or bottom-up? (recap)

Rule of thumb: think recursively, write iteratively.

	Top-down memoization	Bottom-up table filling
Subproblems solved	only those reachable from the start	all of them
Evaluation order	implicit (recursion)	you must choose a valid order
Overheads	recursion, "already computed?" checks	tight loops, cache-friendly
Saving memory	hard	easy when only recent entries are needed

Constructing an optimal solution

```
value[0] = 0;
for (int x = 1; x <= n; x++) {
value[x] = INF;
for (auto c : coins)
if (x-c >= 0 && value[x-c]+1 < value[x]) {
value[x] = value[x-c]+1; first[x] = c;
}
}
while (n > 0) { cout << first[n] << "\\n"; n -=
first[n]; }
```

Store, for each sum x, the first coin of an optimal solution:
Coins {1, 3, 4}, n = 10: prints 3, 3, 4.

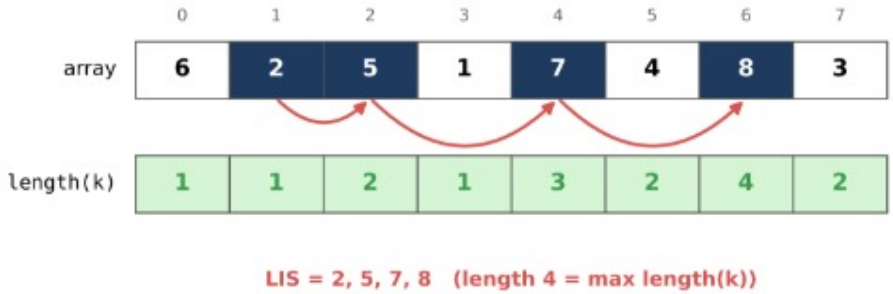
Counting the solutions

```
count[0] = 1;
for (int x = 1; x <= n; x++)
for (auto c : coins)
if (x-c >= 0) { count[x] += count[x-c]; count[x] %= m;
}
```

solve(x) = 0 if x \ 0
Now count the ways to form x. Coins {1, 3, 4}, x = 5 → 6 ways: 1+1+1+1+1, 1+1+3, 1+3+1, 3+1+1, 1+4, 4+1.
The answer grows quickly, so it is usually asked modulo m, e.g. m = 10^9 + 7.
Careful: this counts ordered sequences (3+1+1 ≠ 1+3+1). To count multisets, swap the loops (coins outer, x inner): for x = 5 that gives 3 ways.

Longest increasing subsequence (LIS)

```
for (int k = 0; k < n; k++) {
length[k] = 1;
for (int i = 0; i < k; i++)
if (array[i] < array[k]) length[k] =
max(length[k], length[i]+1);
}
```

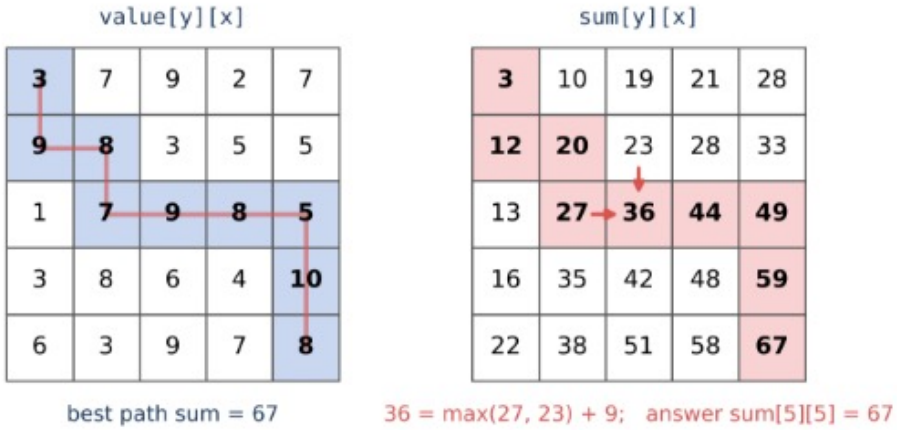


length(k) = 1 + max { length(i) : i < k and array[i] < array[k] } (= 1 if no such i)
Longest sequence of array elements, left to right, each larger than the previous.
length(k) = length of the longest increasing subsequence ending at position k:
$O(n^2)$. An O(n log n) method exists — try to find it.

Array

Paths in a grid

```
for (int y = 1; y <= n; y++)
for (int x = 1; x <= n; x++)
sum[y][x] = max(sum[y][x-1], sum[y-1][x]) +
value[y][x];
```



sum(y, x) = max(sum(y, x−1), sum(y−1, x)) + value[y][x], with sum = 0 outside the grid

n × n grid of positive numbers; move only right or down from the upper-left to the lower-right corner; maximise the sum along the path.

$O(n^2)$ time; the path is recovered by walking back from sum[n][n].

Left: the grid with the optimal path (sum 67). Right: the DP table; each entry comes from its left or upper neighbour

Knapsack: which sums can be formed?

```
possible[0] = true;
for (int k = 1; k <= n; k++)
for (int x = W; x >= 0; x--)
if (possible[x]) possible[x+w[k]] = true;
```



possible(x, k) = possible(x − w\ k, k − 1) ∨ possible(x, k − 1); possible(x, 0) is true only for x = 0

Weights [w_1, …, w\ n]; which sums can be formed from a subset? Weights [1, 3, 3, 5]: every sum 0 … 12 except 2 and 10.

possible(x, k) = can we form x using the first k weights?

O(nW) time, W = total weight. One-dimensional version — scan x from right to left so each weight is used at most once:

possible(x, k) for weights

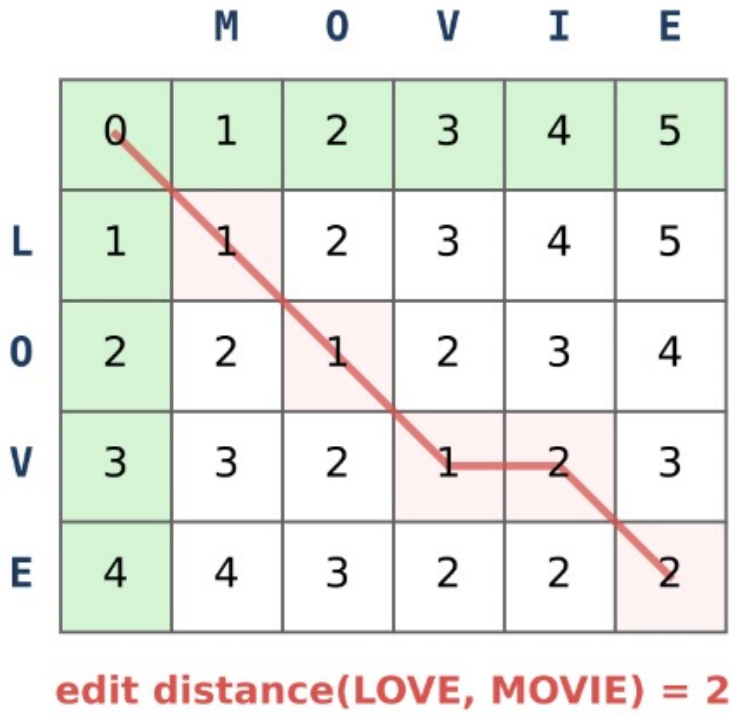
Edit distance

distance(a, b) = min(distance(a, b−1) + 1 [insert], distance(a−1, b) + 1 [remove], distance(a−1, b−1) + cost(a, b) [match / modify]), cost = 0 if x[a] = y[b], else 1

Minimum number of insertions, deletions and substitutions turning string x into string y. LOVE → MOVE → MOVIE: distance 2.

distance(a, b) = edit distance between the prefixes x[0...a] and y[0...b]:

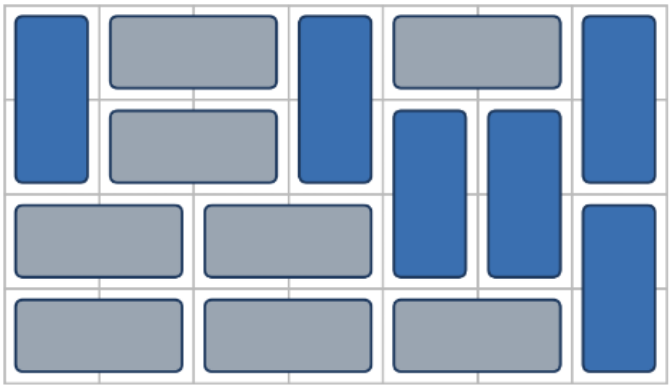
O(nm) time for strings of length n and m.



DP table for LOVE vs MOVIE; the red path is one optimal sequence of edits (modify L→M, match O, V, insert I, match E)

[OPTIONAL] Counting tilings: DP over row profiles

How many ways to tile an n × m grid with 1 × 2 and 2 × 1 dominoes? 4 × 7: 781 ways.
Go row by row. The state is the shape of the current row; a row only has to be compatible with the previous one.
Encode rows as bitmasks: $O(n \cdot 2^{2m})$ time — rotate so that m is the shorter side.
(A closed formula also exists, but its real-number product is awkward to evaluate exactly.)



One of the 781 domino tilings of a 4 × 7 grid

How to design a DP — recipe (recap)

Time ≈ (number of states) × (work per state).

Recipe 1. Write the complete search as a recursive function — what is the first (or last) decision? 2. Identify the state: the parameters of that function (sum x; position k; cell (y, x); prefixes (a, b)). 3. Write the recurrence and base cases. 4. Choose an evaluation order in which every dependency is computed first. 5. Read off the answer; keep a choice table to reconstruct the solution.

Quiz — dynamic programming

QUIZ

1. Coins {1, 5, 6, 9}: what is solve(11)? (Compare with the greedy answer from the previous quiz.)

2. Coins {1, 2}, target 4: how many ordered ways? How many unordered?

3. What is the LIS length of [3, 1, 4, 1, 5, 9, 2, 6]?

4. What is the edit distance between KITTEN and SITTING?

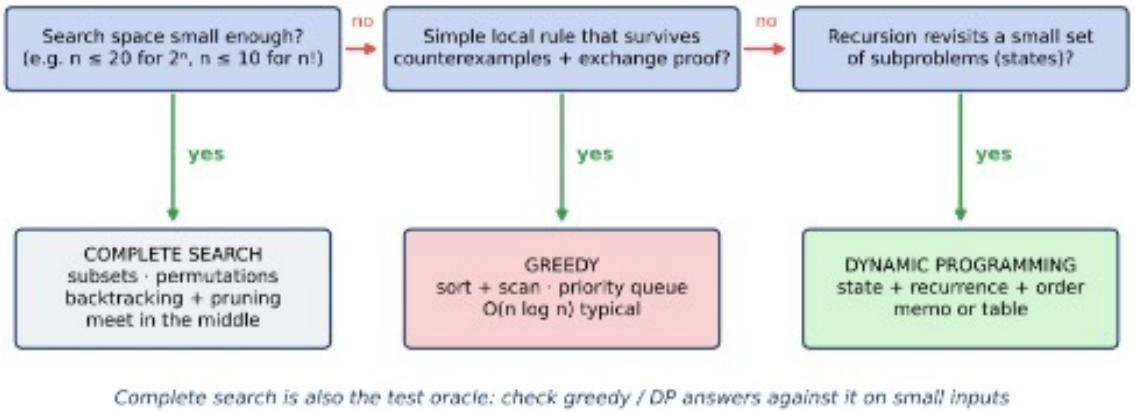
5. Which sums can be formed from weights [2, 3, 7]?

WRAP-UP

Wrap-up

Which technique?

One problem, three answers: coin change by complete search is exponential; greedy is fast but wrong for {1, 3, 4}; DP is O(nk) and always right.



A quick decision guide

Takeaways

1. Check the input size first: $n \leq 20$ often means complete search.
2. Backtracking = do / recurse / undo; prune early and often.
3. Meet in the middle turns 2^n into $2^{n/2}$.
4. Greedy needs proof: counterexamples to reject, exchange arguments to accept.
5. DP = complete search + memoization. Define the state, write the recurrence, fill in a valid order, reconstruct.